PRÁCTICA DE LABORATORIO

Sistema modular de segmentación de clientes

React + operaciones de negocio + algoritmo K-Means

CASO REALISTA: una tienda minorista quiere conocer mejor a sus clientes para diseñar campañas comerciales.

DATOS DE LA PRÁCTICA	DETALLE
Programa	Ingeniería de Software con Inteligencia Artificial
Referencia curricular	Semana 10: aprendizaje no supervisado y algoritmo K-Means
Tecnología de interfaz	React + TypeScript + Vite
Tecnología de análisis	Python con scikit-learn (servicio API opcional para integración)
Modalidad	Individual o equipos de 2–3 estudiantes
Duración sugerida	6 a 8 horas de trabajo

ENTREGABLE: aplicación funcional, código organizado por módulos y evidencia de pruebas.

1. Contexto del caso

La empresa ficticia Comercial Nova vende productos de consumo y registra compras de clientes. Actualmente, el equipo comercial envía la misma promoción a todos, sin distinguir hábitos de compra. La gerencia necesita una aplicación que permita consultar los datos, ejecutar una segmentación y revisar los grupos encontrados para tomar decisiones comerciales.

Problema que resolverás

Construir un sistema web modular en React que permita importar o registrar clientes, visualizar indicadores, ejecutar una segmentación por K-Means, consultar el grupo asignado a cada cliente y preparar una recomendación comercial por segmento. El sistema debe separar responsabilidades en módulos independientes.

2. Relación con los contenidos de la semana 10

Contenido de la semana	Cómo se demuestra en la práctica
Aprendizaje no supervisado	El sistema descubre grupos sin contar con una etiqueta previa de segmento.
Clasificación frente a agrupamiento	El estudiante explica que K-Means agrupa por similitud; no predice una categoría conocida como un clasificador supervisado.
Algoritmo K-Means	Configura K, asigna puntos a centroides y actualiza centroides hasta converger.
Preparación de datos	Valida datos, maneja valores faltantes y escala variables numéricas antes del agrupamiento.
Interpretación de resultados	Describe cada segmento usando promedios de gasto, visitas y antigüedad.

Importante: el documento curricular adjunto sitúa K-Means y aprendizaje no supervisado en la semana 10, con implementación en Python. React será la interfaz del sistema; el cálculo de K-Means puede hacerse con un servicio Python o simularse inicialmente para desarrollar la interfaz.

3. Objetivos de aprendizaje

- Diseñar una aplicación React con módulos independientes y responsabilidades claras.
- Implementar operaciones CRUD para administrar registros de clientes.
- Validar entradas, manejar errores y mostrar estados de carga y resultados.
- Preparar variables numéricas para el algoritmo K-Means y explicar la asignación a centroides.
- Presentar resultados mediante tarjetas KPI, tabla filtrable y visualización de segmentos.
- Integrar la interfaz con una API o separar una capa de servicios para poder conectarla posteriormente.

4. Requerimientos funcionales

ID	Módulo	Requerimiento verificable
RF-01	Panel principal	Mostrar total de clientes, gasto promedio, visitas promedio y número de segmentos detectados.
RF-02	Clientes	Registrar, listar, editar, buscar y eliminar un cliente con confirmación.
RF-03	Datos	Validar campos obligatorios, números no negativos e identificadores únicos.
RF-04	Segmentación	Permitir elegir K entre 2 y 5 y ejecutar el proceso de agrupamiento.
RF-05	Resultados	Mostrar el segmento de cada cliente y un resumen por grupo.
RF-06	Análisis	Filtrar por segmento y ordenar por gasto o visitas.

RF-07	Recomendaciones	Mostrar una propuesta comercial orientativa por segmento.
RF-08	Auditoría básica	Mostrar fecha y hora de la última ejecución y el número de registros procesados.
RF-09	Persistencia	Conservar datos después de recargar, mediante localStorage o una API/base de datos.
RF-10	Experiencia	Mostrar estados vacíos, errores comprensibles, confirmaciones y diseño adaptable.

5. Reglas de negocio

- Cada cliente debe tener un código único, nombre, gasto acumulado, cantidad de visitas y meses como cliente.
- Gasto acumulado, visitas y antigüedad no pueden ser negativos.
- No se ejecutará la segmentación si hay menos registros válidos que el valor K seleccionado.
- K-Means trabajará con variables numéricas; el nombre y el código del cliente no se usan como variables de agrupamiento.
- Como gasto, visitas y antigüedad usan escalas distintas, se deben estandarizar antes de calcular distancias.
- El número de segmento (por ejemplo, 0 o 1) es una etiqueta técnica; el significado comercial se asigna después de revisar los promedios.
- La recomendación comercial es orientativa: el sistema no debe afirmar que un segmento representa una verdad absoluta sobre una persona.
- Antes de eliminar un cliente, la interfaz solicitará confirmación.

6. Datos de prueba

Registra estos datos manualmente o crea un archivo JSON/CSV para importarlos. Las cifras son ficticias y están expresadas para practicar.

Código	Cliente	Gasto acumulado (S/)	Visitas en 6 meses	Antigüedad (meses)
C001	Ana Torres	1800	12	24
C002	Luis Rojas	2200	15	30
C003	Marta Díaz	1600	10	18
C004	José Vega	180	2	3
C005	Rosa León	250	3	5
C006	Pedro Gil	120	1	2
C007	Elena Ruiz	850	7	12
C008	Marco Soto	720	6	10
C009	Sofía Paz	980	8	14
C010	Diego Cruz	400	4	8
C011	Lucía Rey	520	5	9
C012	Raúl Ponce	3000	18	36

Puedes añadir más registros. Prueba también con campos vacíos, valores negativos, códigos duplicados y un conjunto demasiado pequeño para el K seleccionado.

7. Arquitectura modular propuesta

Organiza el proyecto por funciones. No construyas toda la aplicación en un único componente.

Carpeta / archivo sugerido	Responsabilidad
src/app/	Configuración de rutas, layout y navegación.
src/modules/dashboard/	Panel principal y tarjetas de indicadores.
src/modules/customers/	Listado, formulario, búsqueda y operaciones CRUD.
src/modules/segmentation/	Selector K, ejecución, resultados y resumen de grupos.

src/modules/recommendations/	Reglas para sugerencias comerciales según los perfiles encontrados.
src/components/ui/	Componentes reutilizables: botón, modal, campo, tabla, badge y tarjeta.
src/services/customerService.ts	Acceso a datos; inicialmente localStorage o datos simulados.
src/services/segmentationService.ts	Contrato de servicio para solicitar la segmentación a la API.
src/types/	Tipos TypeScript de clientes, segmentos y respuestas de API.
api/ (opcional)	Servicio Python que prepara datos y ejecuta K-Means con scikit-learn.

Modelo mínimo de datos

Customer = { id: string; nombre: string; gastoAcumulado: number; visitas6Meses: number; antiguedadMeses: number; segmento?: number }

SegmentationSummary = { k: number; fechaEjecucion: string; cantidadProcesada: number; grupos: { id: number; cantidad: number; gastoPromedio: number; visitasPromedio: number; antiguedadPromedio: number }[] }

8. Flujo de operaciones del sistema

1. El usuario abre el panel y revisa los indicadores.
2. En Clientes, registra o edita un cliente; la interfaz valida y guarda el cambio.
3. El usuario entra a Segmentación y selecciona K (por ejemplo, 3).
4. El sistema verifica los datos, estandariza las variables y ejecuta K-Means.
5. La respuesta devuelve el grupo asignado a cada cliente y los resúmenes por grupo.
6. El panel actualiza sus indicadores y permite filtrar la tabla por segmento.
7. El usuario interpreta los promedios y revisa recomendaciones comerciales para cada grupo.

9. Desarrollo por etapas

Etapa	Actividades	Criterio de avance
1. Preparación	Crear proyecto React + TypeScript; configurar estilos y navegación.	La aplicación inicia sin errores y muestra el layout.
2. Clientes	Crear tipos, servicio de datos, tabla, formulario y CRUD.	Crear, editar, buscar y eliminar funciona con validaciones.
3. Dashboard	Calcular indicadores desde el conjunto de clientes.	Los KPI cambian al modificar los datos.
4. Segmentación	Implementar selector K, preparación de datos y contrato de servicio.	La interfaz gestiona carga, éxito, datos inválidos y error.
5. K-Means	Integrar API Python o implementar el proceso en un entorno de práctica autorizado.	Se asigna un grupo a cada registro válido y se obtienen centroides/resúmenes.
6. Interpretación	Mostrar conteos, promedios y recomendaciones por segmento.	El estudiante puede justificar el perfil de cada grupo con datos.
7. Pruebas	Ejecutar casos normales y límites; corregir defectos.	Se adjunta evidencia de al menos 8 pruebas.

10. Guía técnica para K-Means

El algoritmo agrupa registros de acuerdo con la cercanía entre sus variables numéricas. En esta práctica se usarán gasto acumulado, visitas y antigüedad.

8. Seleccionar las tres variables numéricas y comprobar que sean válidas.
9. Estandarizar las variables para que una escala grande no domine la distancia.
10. Elegir K, inicializar los centroides y asignar cada punto al centroide más cercano.
11. Actualizar cada centroide con la media de los puntos asignados.

12. Repetir asignación y actualización hasta converger o alcanzar el máximo de iteraciones.
13. Devolver la etiqueta de grupo por cliente y calcular estadísticas descriptivas por grupo.
14. Ejecutar más de una inicialización si la librería lo permite y usar una semilla para facilitar resultados reproducibles.

Pseudocódigo

```
validar(clientes, K)
X = [gastoAcumulado, visitas6Meses, antiguedadMeses]
X_escalado = estandarizar(X)
modelo = KMeans(n_clusters=K, random_state=42, n_init=10)
etiquetas = modelo.fit_predict(X_escalado)
devolver(clientes_con_segmento, centroides, resumen_por_grupo)
```

11. Contrato sugerido de API (opcional)

Elemento	Definición
Método y ruta	POST /api/segmentation/run
Entrada	{ "k": 3, "customers": [{ "id": "C001", "gastoAcumulado": 1800, "visitas6Meses": 12, "antiguedadMeses": 24 }] }
Respuesta	{ "k": 3, "labels": [{ "id": "C001", "segmento": 1 }], "groups": [{ "segmento": 1, "cantidad": 4, "gastoPromedio": 1200, "visitasPromedio": 8, "antiguedadPromedio": 15 }] }
Errores	400 si K o los datos no son válidos; 500 si falla el proceso; el frontend debe mostrar un mensaje entendible.

Si aún no se ha estudiado creación de API, implementa primero segmentationService con datos simulados. Mantén la misma firma para reemplazar el simulador por el backend Python más adelante.

12. Pruebas mínimas obligatorias

N.º	Prueba	Resultado esperado
1	Registrar cliente con todos los datos válidos.	Se guarda y aparece en la tabla.
2	Registrar cliente con código duplicado.	Se rechaza y se informa el motivo.
3	Ingresar gasto negativo.	La validación impide guardar.
4	Editar visitas de un cliente.	Se actualiza el registro y los KPI.
5	Buscar por nombre o código.	La tabla muestra coincidencias.
6	Eliminar un cliente y cancelar confirmación.	El registro se conserva.
7	Ejecutar K=3 con datos suficientes.	Se muestran segmentos y resumen.
8	Elegir K mayor que el número de registros válidos.	Se impide ejecutar y se informa el motivo.
9	Simular fallo de API o ausencia de datos.	Se muestra error o estado vacío sin romper la aplicación.
10	Recargar la página.	Los datos persisten si se implementó persistencia local.

13. Interfaz esperada

Pantalla	Elementos que debe incluir
Panel	Barra lateral o navegación superior, título, 4 KPI y resumen visual de grupos.
Clientes	Buscador, botón Nuevo cliente, tabla con acciones y formulario/modal.
Segmentación	Selector K, botón Ejecutar, estado de carga, fecha de ejecución y resumen de grupos.
Detalle de segmento	Cantidad de clientes, promedios de gasto/visitas/antigüedad y

	tabla filtrada.

Diseño recomendado: fondo claro, azul corporativo para títulos y navegación, cyan para acciones principales, tarjetas con espacio suficiente, etiquetas de estado legibles y tablas adaptables. Prioriza claridad sobre efectos decorativos.

14. Entregables del estudiante

- Código fuente organizado por módulos, con nombres claros y componentes reutilizables.
- Capturas de las pantallas principales y de una segmentación ejecutada.
- Archivo README con instalación, ejecución, estructura y decisiones técnicas.
- Tabla de pruebas con entrada, resultado esperado, resultado obtenido y estado.
- Breve informe: por qué se considera aprendizaje no supervisado, qué variables se usaron, por qué se escalaron y cómo se interpretaron los grupos.
- Si se creó API Python: código del endpoint, dependencias y ejemplo de petición/respuesta.

15. Rúbrica de evaluación

Criterio	Puntaje	Indicador
Arquitectura modular	20	Responsabilidades separadas, componentes reutilizables y servicios desacoplados.
Operaciones de clientes	20	CRUD funcional, validaciones, búsqueda y confirmación.
K-Means y preparación	25	Datos correctos, escalado, ejecución del algoritmo y asignación de grupos.
Visualización e interpretación	15	KPI, resumen por grupo y explicación basada en promedios.
Pruebas y manejo de errores	10	Casos límite, estados de carga/error y evidencia.
Documentación y presentación	10	README, capturas y explicación técnica ordenada.
TOTAL	100	Aprobación sugerida: 70 puntos.

16. Preguntas de análisis

15. ¿Cuál es la diferencia entre clasificación supervisada y agrupamiento no supervisado?
16. ¿Por qué no se debe incluir el nombre del cliente como variable numérica de K-Means?
17. ¿Qué podría ocurrir si no se estandarizan gasto, visitas y antigüedad?
18. ¿Cómo cambia la interpretación cuando se selecciona K=2 en lugar de K=4?
19. ¿Por qué los identificadores de grupo no significan por sí mismos “cliente frecuente” o “cliente premium”?
20. ¿Qué limitaciones tiene usar únicamente tres variables para describir hábitos de compra?
21. ¿Qué responsabilidad cumple React y cuál cumple el servicio Python en la arquitectura propuesta?

17. Checklist antes de entregar

☐ La aplicación arranca y no presenta errores en consola.

☐ Los módulos están separados y el acceso a datos no está mezclado con toda la interfaz.

☐ Las operaciones de clientes funcionan y los datos se validan.

☐ Se puede ejecutar o simular el flujo de segmentación y explicar qué parte calcula K-Means.

☐ Los resultados se entienden y los errores se muestran de forma clara.

☐ Se adjuntan capturas, README y tabla de pruebas.

Referencia curricular utilizada

Documento proporcionado: PIAD-427_CONTENIDO.pdf, Programa de Formación Profesional, módulo Machine Learning y Deep Learning, carrera Ingeniería de Software con Inteligencia Artificial. En la hoja de programación, semana 10 aborda aprendizaje no supervisado, diferencias entre clasificación y agrupamiento, e implementación de K-Means en Python (páginas 3–4 del PDF).

La aplicación de segmentación comercial y la arquitectura React/API son el caso práctico propuesto para conectar esos contenidos con el desarrollo de un sistema modular; no son un proyecto descrito literalmente en el documento curricular.